МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №1

з курсу: „ Інтернет речей та вбудовані системи”

виконав студент 5го курсу
групи КН-51мп
Сидько Н.С.

КИЇВ
2025

Мета роботи: вивчення основних принципів програмування та моделювання мікроконтролерів (МК) у САПР Proteus, отримання практичних навичок з налаштування GPIO, переривань, аналого цифрового перетворювача (АЦП)

Хід роботи

1.1. Створити проект в Proteus на основі МК STM32F103C6.
1.2. До GPIO (лінії введення-виведення) згідно з табл. 1.1 під'єднати світлодіод, резистор з опором 100 Ом та шину живлення +3.3 В.

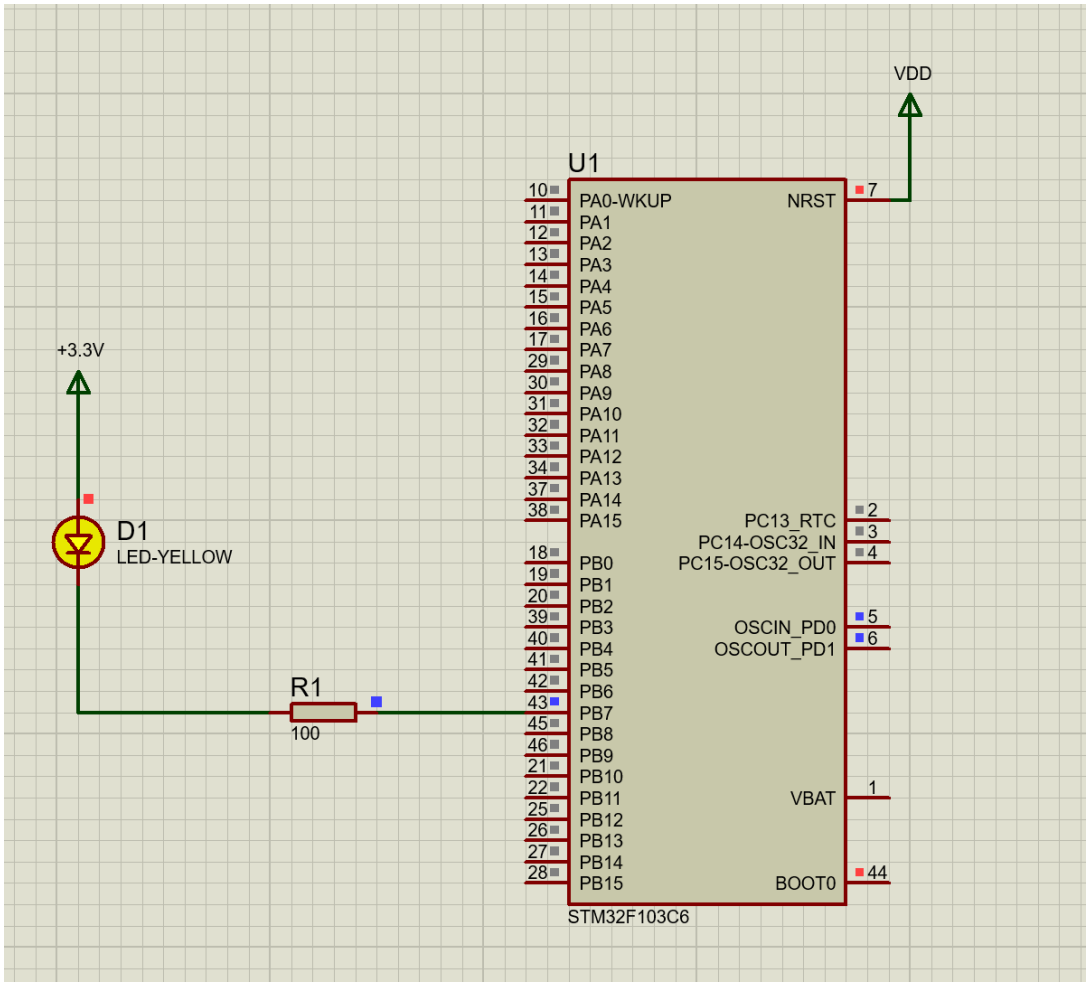
1.2. До GPIO (лінії введення-виведення) згідно з табл. 1.1 під'єднати світлодіод, резистор з опором 100 Ом та шину живлення +3.3 В.

Таблиця 1.1.

№ вар.	GPIO	№ вар.	GPIO	№ вар.	GPIO	№ вар.	GPIO	№ вар.	GPIO
1	PA15	9	PA8	17	PB3	25	PB11	33	PA0
2	PA1	10	PA9	18	PB4	26	PB12	34	PA1
3	PA2	11	PA10	19	PB5	27	PB13	35	PA2
4	PA3	12	PA11	20	PB6	28	PB14	36	PA3
5	PA4	13	PA12	21	PB7	29	PB15	37	PA4
6	PA5	14	PA15	22	PB8	30	PC13	38	PA5
7	PA6	15	PB15	23	PB9	31	PC14	39	PA6
8	PA7	16	PB1	24	PB10	32	PC15	40	PA7

1.3. Створити проект прошивки МК, в якому:
- подати тактування на порт, до складу якого входить GPIO;
- сконфігурувати лінію GPIO на вхід, режим Push-pull з частотою **2 МГц для варіантів N%3 = 0**; 10 МГц для варіантів N%3 = 1; 50 МГц для варіантів N%3 = 2, де N – номер варіанту;
- сформувати низький рівень сигналу на виході GPIO.

1.4. Скомпілювати проект прошивки, запустити модель проекту і пересвідчитись, що протягом симуляції роботи МК світлодіод світиться.



*main.c

```c
#include <stm32f1xx.h>
#include "stm32f103x6.h"

// Функція ініціалізує пін PB7 як вихід з частотою 2 МГц у режимі Push-Pull
static void gpio_pb7_output_pp_2MHz(void) {
    // 1) Увімкнення тактування порту B
    RCC->APB2ENR |= RCC_APB2ENR_IOPBEN;

    // 2) Налаштування PB7: вихід, Push-Pull, швидкість 2 МГц
    //    Кожен пін у CRL займає 4 біти: CNF[1:0] і MODE[1:0]
    //    Очистимо налаштування для PB7
    GPIOB->CRL &= ~(0xF << (7 * 4));
    //    Встановимо MODE = 10 (2 MHz), CNF = 00 (Push-Pull)
    GPIOB->CRL |=  (0x2 << (7 * 4));

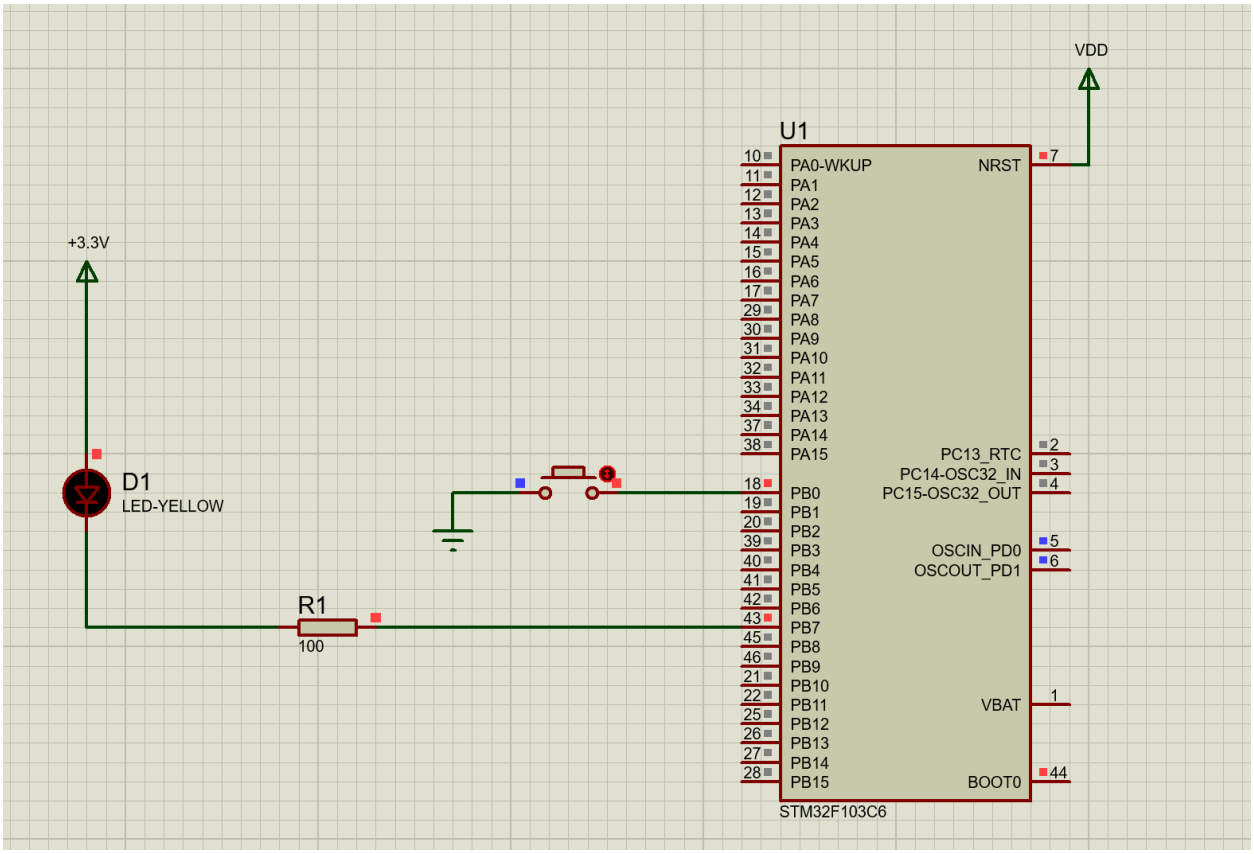
    // 3) Встановлюємо логічний 0 на PB7 → вмикаємо світлодіод
    GPIOB->BRR = (1U << 7);
}

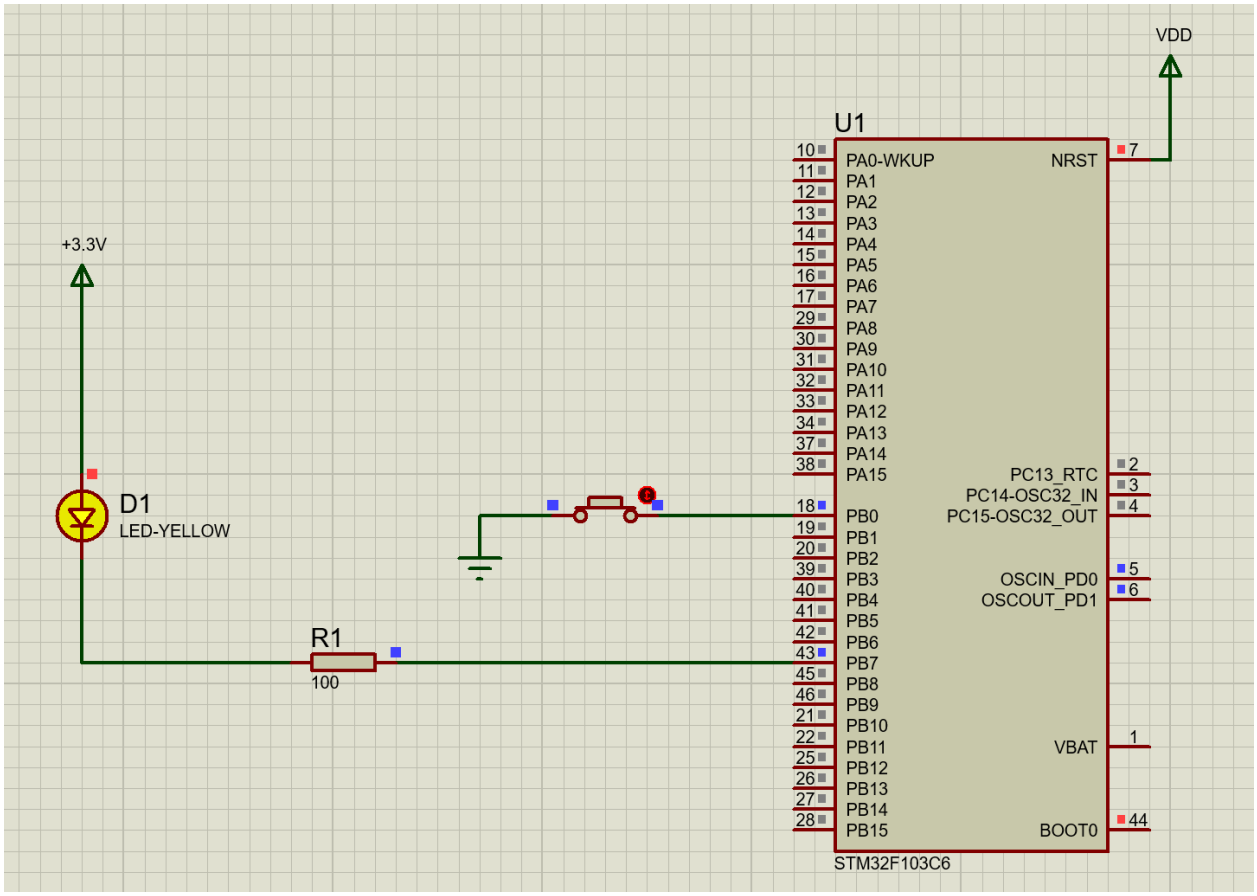
int main(void) {
    // Ініціалізуємо PB7
    gpio_pb7_output_pp_2MHz();

    // Нескінченний цикл (процесор просто очікує, нічого не виконується)
    while (1) {
        __NOP();  // Інструкція "нічого не робити" (No Operation)
    }
}
```

1.5. До GPIO під’єднати кнопку та резистор з опором 5 кОм (парний варіант – до GPIO PA0, непарний – до PB0).

1.6. До коду проекту додати:
- тактування на порт, до складу якого входить GPIO;
- сконфігурувати лінію GPIO на вхід, режим Pull-up
- модифікувати програму таким чином, щоб при натисненні кнопки загорявся світлодіод, а при відпусканні – гаснув. Для цього у нескінченому циклі сканується значення біту регістра IDR, лінії налаштованої на вхід. Залежно від його значення, формується значення вихідної GPIO.





*main.c

```c
#include "stm32f1xx.h"

int main(void) {
    // 1. Увімкнення тактування порту GPIOB
    RCC->APB2ENR |= RCC_APB2ENR_IOPBEN;

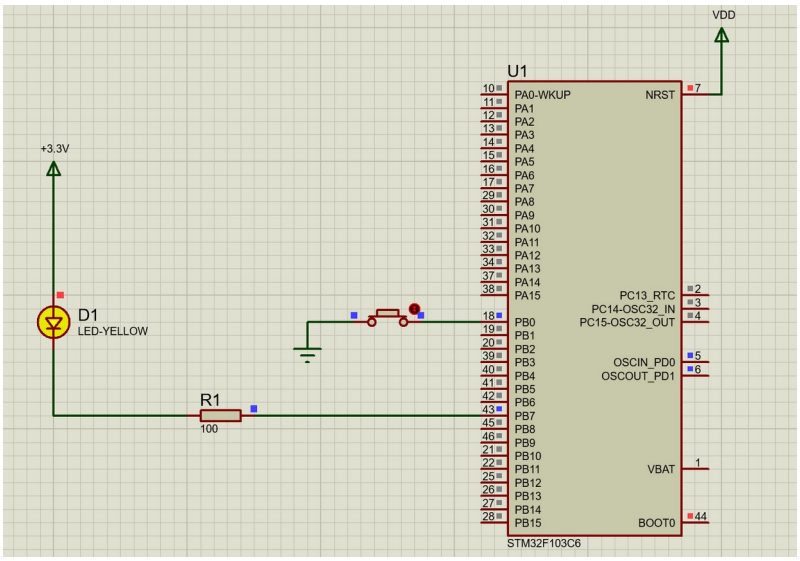
    // 2. Налаштування PB7 як вихід у режимі Push-pull, з частотою 2 МГц
    GPIOB->CRL &= ~(GPIO_CRL_MODE7 | GPIO_CRL_CNF7); // Очистити налаштування MODE7[1:0] і CNF7[1:0]
    GPIOB->CRL |= GPIO_CRL_MODE7_1;                  // MODE7 = 0b10 (2 MHz), CNF7 = 0b00 (Push-pull)

    // 3. Налаштування PB0 як вхід з внутрішнім підтягуванням (pull-up)
    GPIOB->CRL &= ~(GPIO_CRL_MODE0 | GPIO_CRL_CNF0); // Очистити попередні налаштування
    GPIOB->CRL |= GPIO_CRL_CNF0_1;                   // CNF0 = 0b10 (Input with pull-up / pull-down)
    GPIOB->ODR  |= GPIO_ODR_ODR0;                    // Встановити підтяжку до Vcc (ODR=1 → pull-up)

    // 4. Стартово вимикаємо світлодіод (PB7 = 1 → LED OFF, якщо LED з'єднаний анодом на живлення)
    GPIOB->BSRR = GPIO_BSRR_BS7;

    // 5. Нескінченний цикл опитування кнопки
    while (1) {
        // Якщо кнопка натиснута (PB0 = 0)
        if ((GPIOB->IDR & GPIO_IDR_IDR0) == 0) {
            GPIOB->BRR = GPIO_BRR_BR7;  // PB7 = 0 → LED ON
        } else {
            // Якщо кнопка відпущена (PB0 = 1)
            GPIOB->BSRR = GPIO_BSRR_BS7; // PB7 = 1 → LED OFF
        }
    }
}
```

1.7. Дозволити переривання GPIO. Налаштувати переривання на вхідну лінію GPIO. Модифікувати програму таким чином, щоб керування світлодіодом здійснювалось з обробника переривання лінії, налаштованої на вхід.



*main.c

```c
#include "stm32f1xx.h"

// Обробник переривання для лінії EXTI0 (підключено до PB0)
void EXTI0_IRQHandler(void) {
    // Перевірка, чи активне переривання на EXTI0
    if (EXTI->PR & EXTI_PR_PR0) {
        // Зчитування стану кнопки на PB0 і керування LED на PB7
        if ((GPIOB->IDR & GPIO_IDR_IDR0) == 0) {
            // Якщо PB0 = 0 (натиснуто), вмикаємо LED (PB7 = 0)
            GPIOB->BRR = GPIO_BRR_BR7;
        } else {
            // Якщо PB0 = 1 (відпущено), вимикаємо LED (PB7 = 1)
            GPIOB->BSRR = GPIO_BSRR_BS7;
        }

        // Скидання прапорця переривання (очищення pending)
        EXTI->PR = EXTI_PR_PR0;
    }
}

int main(void) {
    // --- Увімкнення тактування для GPIOB і AFIO ---
    RCC->APB2ENR |= RCC_APB2ENR_IOPBEN | RCC_APB2ENR_AFIOEN;

    // --- Налаштування PB7 як вихід, Push-pull, 2 MHz ---
    GPIOB->CRL &= ~(GPIO_CRL_MODE7 | GPIO_CRL_CNF7); // Очистити налаштування
    GPIOB->CRL |= GPIO_CRL_MODE7_1;                  // MODE7 = 0b10 (2 MHz), CNF7 = 0b00 (Push-pull)
    GPIOB->BSRR = GPIO_BSRR_BS7;                     // Вимкнути LED (PB7 = 1)

    // --- Налаштування PB0 як вхід з підтяжкою pull-up ---
    GPIOB->CRL &= ~(GPIO_CRL_MODE0 | GPIO_CRL_CNF0); // Очистити налаштування
    GPIOB->CRL |= GPIO_CRL_CNF0_1;                   // CNF0 = 0b10 (Input with pull-up/down)
    GPIOB->ODR |= GPIO_ODR_ODR0;                     // Встановити підтягування до Vcc (pull-up)

    // --- Прив'язка EXTI0 до порту PB0 ---
    AFIO->EXTICR[0] &= ~AFIO_EXTICR1_EXTI0;          // Очистити EXTICR1
    AFIO->EXTICR[0] |= AFIO_EXTICR1_EXTI0_PB;        // Прив'язати EXTI0 до PB0

    // --- Налаштування EXTI ---
    EXTI->IMR  |= EXTI_IMR_MR0;      // Дозволити переривання від EXTI0
    EXTI->RTSR |= EXTI_RTSR_TR0;     // Дозволити переривання по зростанню (відпускання кнопки)
    EXTI->FTSR |= EXTI_FTSR_TR0;     // Дозволити переривання по спаду (натискання кнопки)
    EXTI->PR    = EXTI_PR_PR0;       // Очистити прапорець переривання

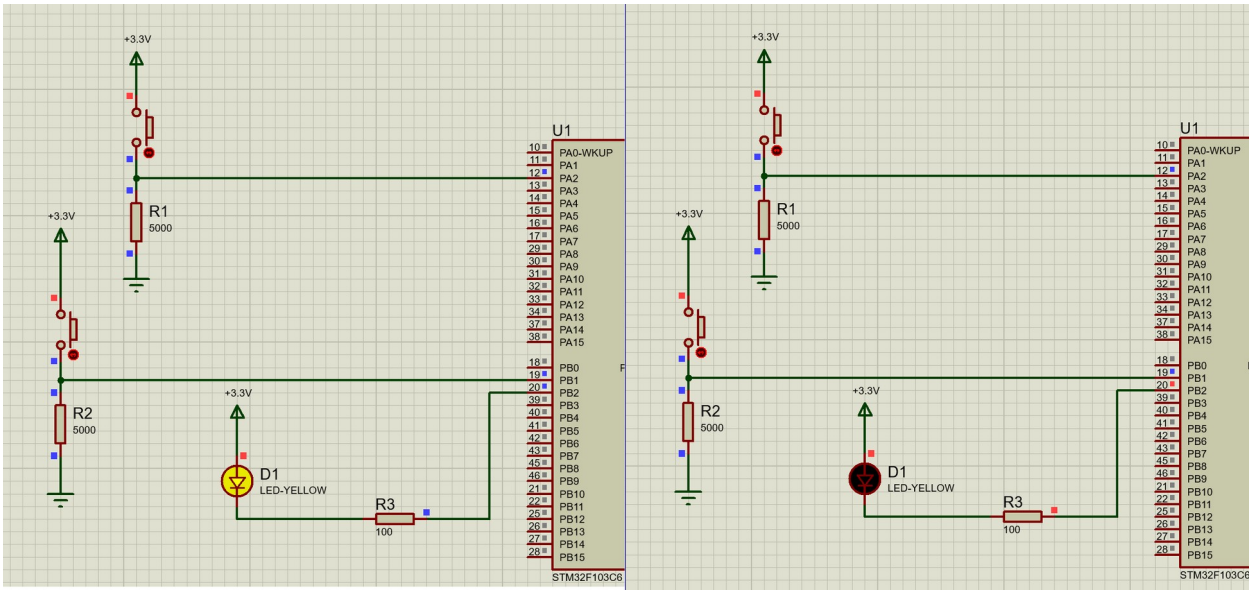
    // --- Увімкнення EXTI0 у контролері переривань NVIC ---
    NVIC_EnableIRQ(EXTI0_IRQn);

    // --- Основний цикл ---
    while (1) {
        __WFI(); // Очікування переривання (режим енергозбереження)
    }
}
```

1.8. Створити новий проект до проекту додати світлодіод та дві кнопки з резисторами. Написати програму на основі обробників переривань від двох кнопок, яка б по натисненню однієї кнопки запалювала світлодіод, іншої – гасила. Кнопки і світлодіод під’єднати до GPIO, вказаних у табл. 1.2.

Таблиця 1.2.

№ вар.	Кнопка1	Кнопка2	Світлодіод	№ вар.	Кнопка1	Кнопка2	Світлодіод
1	PA0	PB1	PB2	21	PA2	PB1	PB2
2	PA0	PB2	PB1	22	PA2	PB0	PB1
3	PA0	PB3	PB1	23	PA2	PB3	PB1
4	PA0	PB4	PB1	24	PA2	PB4	PB1
5	PA0	PB5	PB1	25	PA2	PB5	PB1
6	PA0	PB11	PB1	26	PA2	PB11	PB1
7	PA0	PB12	PB1	27	PA2	PB12	PB1
8	PA0	PB13	PB1	28	PA2	PB13	PB1
9	PA0	PB14	PB1	29	PA2	PB14	PB1
10	PA0	PB15	PB1	30	PA2	PB15	PB1
11	PA1	PB0	PB2	31	PA3	PB0	PB2
12	PA1	PB2	PB1	32	PA3	PB1	PB2



*main.c

```c
#include "stm32f1xx.h"

// --- Обробник переривання від PA2 (EXTI2) ---
void EXTI2_IRQHandler(void) {
    if (EXTI->PR & EXTI_PR_PR2) {
        GPIOB->BRR = GPIO_BRR_BR2;      // Встановити PB2 = 0 → увімкнути LED
        EXTI->PR = EXTI_PR_PR2;         // Скинути прапорець переривання EXTI2
    }
}

// --- Обробник переривання від PB1 (EXTI1) ---
void EXTI1_IRQHandler(void) {
    if (EXTI->PR & EXTI_PR_PR1) {
        GPIOB->BSRR = GPIO_BSRR_BS2;    // Встановити PB2 = 1 → вимкнути LED
        EXTI->PR = EXTI_PR_PR1;         // Скинути прапорець переривання EXTI1
    }
}

int main(void) {
    // --- Увімкнути тактування портів GPIOA, GPIOB і AFIO ---
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPBEN | RCC_APB2ENR_AFIOEN;

    // --- Налаштувати PB2 як вихід (Push-pull), 2 MHz ---
    GPIOB->CRL &= ~(GPIO_CRL_MODE2 | GPIO_CRL_CNF2); // Очистити налаштування
    GPIOB->CRL |= GPIO_CRL_MODE2_1;                  // MODE2 = 0b10 (2 MHz), CNF2 = 0b00 (Push-pull)
    GPIOB->BSRR = GPIO_BSRR_BS2;                     // Стартово вимкнути LED (PB2 = 1)

    // --- Налаштувати PA2 як вхід з підтягуванням до Vcc (pull-up) ---
    GPIOA->CRL &= ~(GPIO_CRL_MODE2 | GPIO_CRL_CNF2);
    GPIOA->CRL |= GPIO_CRL_CNF2_1;                   // CNF2 = 0b10 (input with PU/PD)
    GPIOA->ODR |= GPIO_ODR_ODR2;                     // Встановити підтяжку до Vcc (pull-up)

    // --- Налаштувати PB1 як вхід з підтягуванням до Vcc (pull-up) ---
    GPIOB->CRL &= ~(GPIO_CRL_MODE1 | GPIO_CRL_CNF1);
    GPIOB->CRL |= GPIO_CRL_CNF1_1;                   // CNF1 = 0b10 (input with PU/PD)
    GPIOB->ODR |= GPIO_ODR_ODR1;

    // --- Прив'язати EXTI2 до PA2 ---
    AFIO->EXTICR[0] &= ~AFIO_EXTICR1_EXTI2;          // Очистити EXTICR для EXTI2
    AFIO->EXTICR[0] |= AFIO_EXTICR1_EXTI2_PA;        // Призначити EXTI2 на PA2

    // --- Прив'язати EXTI1 до PB1 ---
    AFIO->EXTICR[0] &= ~AFIO_EXTICR1_EXTI1;          // Очистити EXTICR для EXTI1
    AFIO->EXTICR[0] |= AFIO_EXTICR1_EXTI1_PB;        // Призначити EXTI1 на PB1

    // --- Налаштувати EXTI ---
    EXTI->IMR  |= EXTI_IMR_MR2 | EXTI_IMR_MR1;       // Увімкнути маску для EXTI1 і EXTI2
    EXTI->FTSR |= EXTI_FTSR_TR2 | EXTI_FTSR_TR1;     // Дозволити спрацювання по спаду сигналу

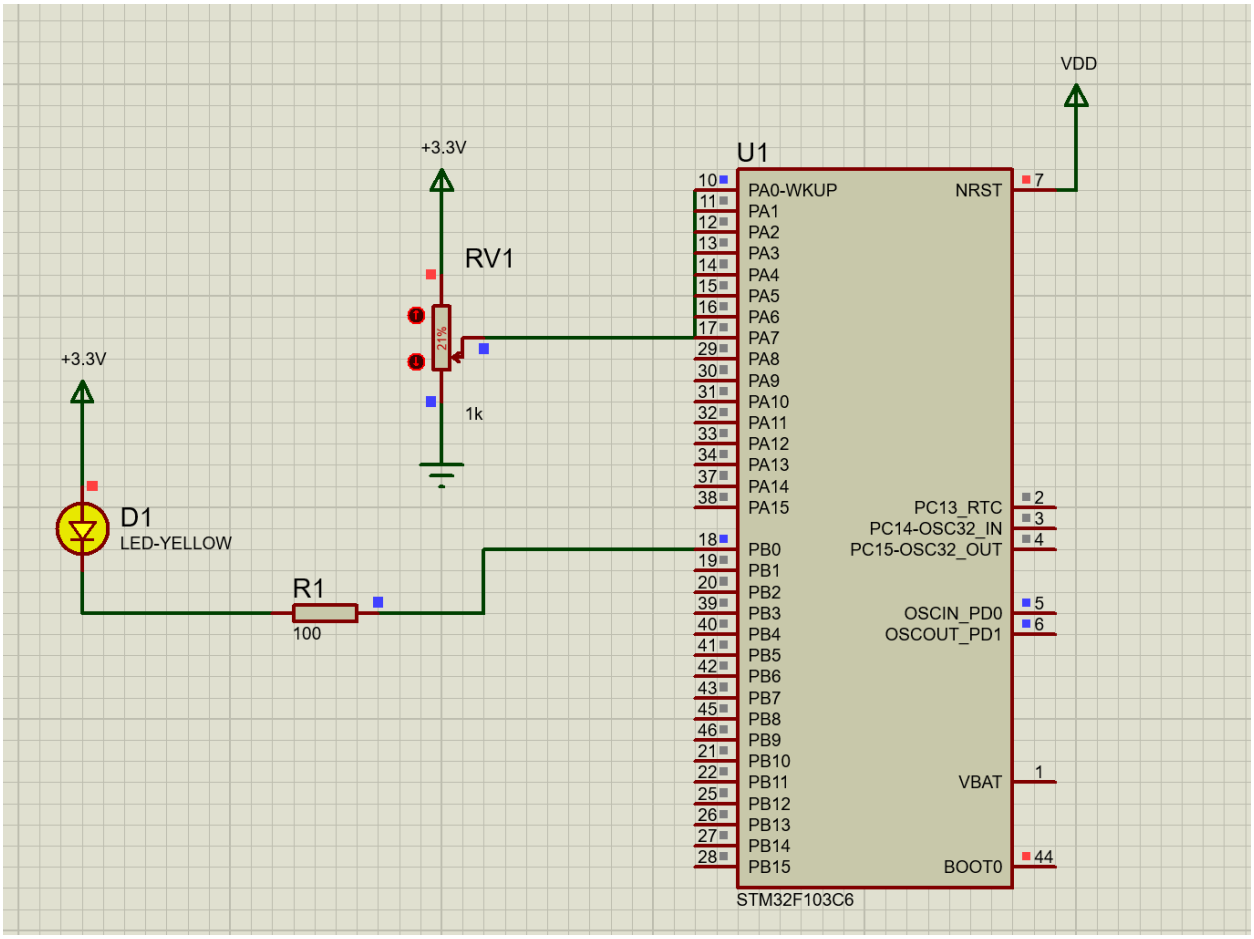
    // --- Увімкнути переривання у NVIC ---
    NVIC_EnableIRQ(EXTI2_IRQn);     // Увімкнути переривання EXTI2 (PA2 → LED ON)
    NVIC_EnableIRQ(EXTI1_IRQn);     // Увімкнути переривання EXTI1 (PB1 → LED OFF)

    while (1) {
        __WFI(); // Очікувати переривання (режим енергозбереження)
    }
}
```

1.9 Створити новий проєкт, налаштувати канал АЦП згідно табл 1.3 на режим Single. Додати до проекту світлодіод з резистором та шину живлення Power, яка під'єднується на вхід АЦП. Написати програму моргання світлодіодом з періодом, який задається виміряним АЦП значенням у мілісекундах.

Таблиця 1.3.

№ вар.	GPIO	№ вар.	GPIO	№ вар.	GPIO	№ вар.	GPIO	№ вар.	GPIO
1	IN0	9	IN8	17	IN6	25	PB11	33	IN4
2	IN1	10	IN9	18	IN7	26	PB12	34	IN5
3	IN2	11	IN0	19	IN8	27	PB13	35	IN6
4	IN3	12	IN1	20	IN9	28	PB14	36	IN7
5	IN4	13	IN2	21	IN0	29	PB15	37	IN8
6	IN5	14	IN3	22	IN1	30	PC13	38	IN9
7	IN6	15	IN4	23	IN2	31	PC14	39	IN0
8	IN7	16	IN5	24	IN3	32	PC15	40	IN1



*main.c

```c
#include "stm32f1xx.h"

// Затримка в тактах (NOP = No Operation)
static void delay(volatile uint32_t ticks) {
    while (ticks--) __NOP();
}

int main(void) {
    // --- Увімкнення тактування портів GPIOA, GPIOB і ADC1 ---
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN     // GPIOA
                 |  RCC_APB2ENR_IOPBEN     // GPIOB
                 |  RCC_APB2ENR_ADC1EN;    // ADC1

    // --- Налаштування PB0 як вихід Push-Pull, 2 МГц ---
    GPIOB->CRL &= ~(GPIO_CRL_MODE0 | GPIO_CRL_CNF0); // Очистити налаштування
    GPIOB->CRL |= GPIO_CRL_MODE0_1;                  // MODE0 = 0b10 (2 MHz), CNF0 = 0b00 (Push-pull)
    GPIOB->BSRR = GPIO_BSRR_BS0;                     // PB0 = 1 (LED вимкнено)

    // --- Налаштування PA0 як аналоговий вхід (ADC1_IN0) ---
    GPIOA->CRL &= ~(GPIO_CRL_MODE0 | GPIO_CRL_CNF0); // MODE0=00, CNF0=00 (аналоговий режим)

    // --- Налаштування ADC1 ---
    RCC->CFGR |= RCC_CFGR_ADCPRE_DIV6;     // Поділ частоти: 72 МГц / 6 = 12 МГц

    ADC1->CR2 = ADC_CR2_ADON;              // Увімкнути ADC
    ADC1->CR2 |= ADC_CR2_RSTCAL;           // Запустити скидання калібрування
    while (ADC1->CR2 & ADC_CR2_RSTCAL);    // Очікування завершення
    ADC1->CR2 |= ADC_CR2_CAL;              // Запустити калібрування
    while (ADC1->CR2 & ADC_CR2_CAL);       // Очікування завершення

    // --- Налаштування каналу IN0 (PA0) ---
    ADC1->SQR1 = 0;                        // Один канал
    ADC1->SQR3 = 0;                        // Перший в черзі — канал 0 (IN0)
    ADC1->SMPR2 |= ADC_SMPR2_SMP0_1 | ADC_SMPR2_SMP0_0; // Зразок 55.5 циклів (оптимально)

    // --- Увімкнути одиничний запуск (Single Conversion) ---
    ADC1->CR2 &= ~ADC_CR2_CONT;            // Вимкнути безперервний режим
    ADC1->CR2 |= ADC_CR2_EXTSEL;           // Джерело запуску — SWSTART
    ADC1->CR2 |= ADC_CR2_EXTTRIG;          // Дозвіл на зовнішній запуск

    while (1) {
        // --- Запуск перетворення ---
        ADC1->CR2 |= ADC_CR2_SWSTART;

        // --- Очікування завершення ---
        while (!(ADC1->SR & ADC_SR_EOC));
        uint16_t adc = ADC1->DR;           // Зчитування результату з ADC

        // --- Розрахунок затримки на основі ADC ---
        uint32_t ticks = 50000 + ((uint32_t)adc * 200000) / 4095;

        // --- Моргати світлодіодом ---
        GPIOB->BRR = GPIO_BRR_BR0;         // PB0 = 0 → LED ON
        delay(ticks);                      // Очікування (тривалість залежить від ADC)
        GPIOB->BSRR = GPIO_BSRR_BS0;       // PB0 = 1 → LED OFF
        delay(ticks);
    }
}
```

1.10. Змінити режим роботи АЦП на Continious.

*main.c

```c
#include "stm32f1xx.h"
// Функція затримки, яка використовується для моргання світлодіода
static void delay(volatile uint32_t ticks) {
    while (ticks--) __NOP();
}
int main(void) {
    // --- Увімкнення тактування портів GPIOA, GPIOB і ADC1 ---
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN     // Порт A
                 |  RCC_APB2ENR_IOPBEN     // Порт B
                 |  RCC_APB2ENR_ADC1EN;    // АЦП1

    // --- Налаштування PB0 як вихід Push-Pull, 2 МГц ---
    GPIOB->CRL &= ~(GPIO_CRL_MODE0 | GPIO_CRL_CNF0); // Очистити конфігурацію
    GPIOB->CRL |= GPIO_CRL_MODE0_1;                  // MODE0 = 0b10 (2 MHz), CNF0 = 0b00 (Push-Pull)
    GPIOB->BSRR = GPIO_BSRR_BS0;                     // Встановити PB0 у 1 (LED вимкнено)

    // --- Налаштування PA0 як аналоговий вхід (ADC1_IN0) ---
    GPIOA->CRL &= ~(GPIO_CRL_MODE0 | GPIO_CRL_CNF0); // MODE0=00, CNF0=00 → Analog mode

    // --- Налаштування частоти АЦП ---
    RCC->CFGR |= RCC_CFGR_ADCPRE_DIV6;               // 72 МГц / 6 = 12 МГц

    // --- Увімкнення ADC та ініціалізація калібрування ---
    ADC1->CR2 = ADC_CR2_ADON;                        // Увімкнути ADC
    ADC1->CR2 |= ADC_CR2_RSTCAL;                     // Скидання калібрування
    while (ADC1->CR2 & ADC_CR2_RSTCAL);              // Очікування завершення
    ADC1->CR2 |= ADC_CR2_CAL;                        // Запуск калібрування
    while (ADC1->CR2 & ADC_CR2_CAL);                 // Очікування завершення

    // --- Налаштування ADC на один канал (канал 0: PA0) ---
    ADC1->SQR1 = 0;                                  // Один канал
    ADC1->SQR3 = 0;                                  // Канал 0 — IN0 (PA0)
    ADC1->SMPR2 |= ADC_SMPR2_SMP0_1 | ADC_SMPR2_SMP0_0; // 55.5 циклів на семплінг (рекомендовано)

    // --- Перехід у режим безперервного перетворення ---
    ADC1->CR2 |= ADC_CR2_CONT;                       // Ввімкнути режим Continuous
    ADC1->CR2 |= ADC_CR2_EXTSEL;                     // Джерело запуску — SWSTART
    ADC1->CR2 |= ADC_CR2_EXTTRIG;                    // Дозвіл запуску
    ADC1->CR2 |= ADC_CR2_ADON;                       // Повторне вмикання для запуску

    while (1) {
        // --- Очікування завершення перетворення ---
        while (!(ADC1->SR & ADC_SR_EOC));            // Очікування флагу завершення
        uint16_t adc = ADC1->DR;                     // Зчитування значення ADC

        // --- Перетворення ADC у затримку ---
        uint32_t ticks = 50000 + ((uint32_t)adc * 200000) / 4095;

        // --- Моргати світлодіодом ---
        GPIOB->BRR = GPIO_BRR_BR0;  // PB0 = 0 → LED ON
        delay(ticks);
        GPIOB->BSRR = GPIO_BSRR_BS0; // PB0 = 1 → LED OFF
        delay(ticks);
    }
}
```

Висновок

У ході лабораторної роботи було ознайомлено з базовими принципами налаштування та програмування мікроконтролера STM32F103C6. Було реалізовано керування світлодіодами за допомогою GPIO-виходів у різних режимах (вивід, вхід з підтяжкою, переривання), а також вивчено роботу аналогово-цифрового перетворювача (АЦП) у режимах одиночного та безперервного перетворення. На основі сигналу з АЦП створено змінний період блимання світлодіода. Отримано практичні навички роботи з регістрами, налаштуванням портів вводу/виводу та системою переривань, що є важливою основою для розробки вбудованих систем на базі STM32.